

Task 2 — QA Strategy & Thinking

The scenario: I've just joined a fintech startup as a QA Engineer. Day one: a mobile trading app for iOS and Android, two weeks from its first public release. No existing test suite, no QA documentation, dev team has been shipping fast. Real user funds are involved.

1. Where do you start?

I don't start by writing tests — I start by figuring out where the money is, because that's where the risk is. Concretely, in the first day or two:

1. **Risk-rank the app, not feature-by-feature but by consequence.** What's the worst realistic outcome? A wrong balance shown, a duplicated order, a withdrawal that goes to the wrong place, unauthorized account access. Those rank far above a mistimed animation or a typo in the FAQ, and the two-week clock means I can't treat everything as equally important.
2. **Map the critical user journeys end to end:** sign-up/KYC, funding an account, placing and cancelling an order, viewing balance/portfolio, withdrawing, session expiry/logout, and whatever push notifications drive re-engagement (price alerts, order fills). These become the spine of everything I test later.
3. **Mine what already exists before assuming nothing does** — crash reports (Crashlytics/Sentry), analytics on real usage, existing Slack threads or tickets about known flaky areas, the API/Postman collection if one exists. "No QA documentation" doesn't mean no signal; devs usually know exactly which corner of the app they're nervous about, and that's often the fastest, most honest risk assessment available.
4. **Get environment access sorted immediately:** staging vs. prod parity, sandboxed payment rails, seeded test accounts with fake funds, API credentials. This is a blocking dependency and the biggest single time sink if left until later.
5. **Establish severity/priority definitions and a bug-triage rhythm before writing a single automated test.** With two weeks and a team "shipping fast," bugs will surface immediately — if there's no shared language for "this blocks release" vs. "this is a polish item," triage itself becomes the bottleneck.
6. **Do a fast, unstructured exploratory pass on both platforms on day one.** Not a test plan yet — just using the app like a real user with real money would, on both iOS and Android. This surfaces the obvious blockers immediately, while there's still time to fix them, instead of discovering them during a "proper" test cycle in week two.

2. How would you approach testing this app?

Risk-based and layered, sized to two weeks — I'd rather have strong coverage on the flows that touch money than shallow coverage everywhere:

- **Manual exploratory testing first**, on the critical journeys above, on both platforms. It's the fastest way to find real bugs when there's zero existing coverage, and it tells me where automation effort is actually worth spending.
- **API/contract-level testing next, before broad UI automation.** Order placement, balance retrieval, auth, deposit/withdrawal endpoints — this is where "is the money right" bugs actually live, and API tests are faster to write, faster to run, and far more stable than driving the same checks through the UI. I'd rather have 50 solid API tests than 10 flaky UI tests covering the same ground.
- **A thin, high-value layer of E2E UI automation** on the flows that are both critical and stable: login, place a simple market order, see the balance update, log out. Not exhaustive UI coverage — the timeline doesn't support it, and UI tests are the most expensive to write and maintain.
- **Money-specific correctness checks:** decimal precision and rounding on every displayed and calculated value, currency formatting, idempotency of order submission (does double-tapping "Buy" place one order or two?), race conditions between "cancel" and "fill," timestamp/timezone consistency on trade history.
- **Device/OS prioritization by real-world usage**, not exhaustive coverage — top 2 iOS versions, top Android OEM/OS combinations from analytics if available, otherwise industry-standard defaults. A cloud device farm (BrowserStack/Firebase Test Lab) for the long tail rather than trying to own physical devices for everything.
- **Non-functional basics:** session/token handling and sensitive-data storage (is anything sensitive logged or cached in plaintext?), behavior under real-world flaky network conditions (does a dropped connection mid-order ever leave an ambiguous state?), basic accessibility.
- **App Store / Play Store compliance, checked early, not at submission.** Apple in particular applies extra scrutiny to financial/trading apps (licensing disclosures, required disclaimers). A rejection discovered on submission day, a week before release, is a self-inflicted crisis.
- **A full-team bug bash a few days before the release candidate is cut**, specifically hunting for anything the structured passes missed.

3. What does QA look like inside a sprint, from ticket creation through to regression?

QA shows up at the start of the ticket, not just at the end of the build:

- **At refinement/ticket creation:** I review acceptance criteria for testability and push on edge cases before a line of code is written — "what happens if the balance is insufficient," "what happens if the session expires mid-submit," "what happens offline." Catching a missing edge case here costs minutes; catching it after merge costs a regression cycle.
- **During development:** I write or update test cases in parallel with the build, not after. For anything touching order execution or balances, API-level tests can often be written against a mocked or staging endpoint before the UI even exists, so verification starts the moment a build is available rather than after.
- **At PR stage:** CI runs unit tests, API/contract tests, and the tagged @smoke E2E suite automatically on every PR. A PR touching a critical flow doesn't merge on a red smoke run.

- **At QA verification:** I test the actual acceptance criteria on a real build (TestFlight / internal test track), plus a short exploratory pass around the change — regressions hide next to the change, not just inside it.
- **Definition of done** includes: acceptance criteria met, an automated test added or updated for the change, no open P1/P2 against it, verified on both platforms. A ticket isn't done because the code compiles.
- **Regression:** every merge to the release branch runs the `@smoke` suite immediately, and the full `@regression` suite runs nightly and again against every release candidate. Any new automated test added during the sprint becomes a permanent part of that regression suite — today's bug fix is tomorrow's regression guardrail.
- **Triage cadence:** new bugs get a severity/priority in the same day they're filed, surfaced in standup — with two weeks on the clock, a bug sitting untriaged for two days is functionally a bug nobody decided to ship.

4. What does your ideal regression suite look like?

Layered by speed and stability, so the fastest, most reliable signal runs most often:

- **Unit tests (dev-owned)** — business logic, fee/PnL calculations, rounding, input validation. Run on every commit; this is where money-math bugs should be caught first, cheapest.
- **API/contract tests** — auth, order placement/cancellation, balance and transaction history, deposit/withdrawal, notification triggers. Run on every PR and before every release. This layer carries the most weight for a trading app, because it verifies correctness without UI flakiness in the way.
- **E2E UI smoke suite (`@smoke`)** — a tight 10–15 minute run of the critical path only (login → view market → place an order → balance updates → logout) on both iOS and Android. Runs on every merge to the release branch. This is the suite that has to stay green, always.
- **E2E UI full regression (`@regression`)** — broader coverage: all navigation, all order types, negative/edge paths, secondary flows. Runs nightly and against every release candidate, on a real device matrix (cloud device farm, not just simulators — simulator-only coverage misses real-device issues like biometric prompts, OS-level permission dialogs, and OEM-specific rendering).
- **Non-functional regression, run on a schedule rather than every PR** — performance benchmarks, basic accessibility scans, and a security checklist (certificate pinning still active, no debug builds/flags leaking into a release artifact).
- **Data-driven, not single-example.** The order-placement flow runs across multiple asset pairs, order types, and amounts — including boundary values like minimum trade size and maximum decimal precision — instead of one hardcoded happy-path case that looks like coverage but isn't.
- **A firm flaky-test policy.** An intermittently failing test on a financial flow gets quarantined and fixed within the sprint, not retired into silence — a flaky test on the order-placement path is worse than no test, because it teaches the team to ignore red.
- **A release gate:** the regression suite must be fully green, or every red test explicitly triaged and waived by the QA lead with a written reason, before a release candidate is approved. No unexplained red goes to the store.

5. What would keep you up at night about this app specifically and releasing to the public?

- **Money correctness under concurrency.** A double-tap on "Buy" or a retried network request producing two orders instead of one; a race between a cancel request and a fill landing in the wrong order and leaving the user's position or balance wrong. These are exactly the bugs that don't show up in a single manual pass and only surface under real concurrent load.
- **Silent ambiguity on poor connectivity.** A user taps "buy," loses signal, and the app doesn't clearly show pending vs. failed vs. succeeded — they either believe they lost money that's actually fine, or they retry and end up with a duplicate order. For a trading app this is a trust-ending bug, not a cosmetic one.
- **Balance/ledger drift** — what's displayed not matching the real ledger after a burst of activity, a backgrounded app mid-transaction, or an app relaunch. This is the single bug category most likely to generate support tickets and regulatory attention simultaneously.
- **No existing regression safety net.** With no test suite going in, every last-minute fix in the final two weeks — and there will be several — is effectively untested beyond manual eyes, which is exactly when regressions are most likely and least likely to be caught before submission.
- **App Store rejection risk, discovered too late.** Financial/trading apps get extra review scrutiny (licensing disclosures, required disclaimers). A rejection found on submission day can cost most of a week just in review turnaround, on a two-week runway.
- **No kill switch.** If something critical ships anyway, is there a remote feature flag to disable trading (or a specific order type) without an emergency app-store resubmission that itself takes days to clear review? If not, that's the first gap I'd close.
- **Device/OS fragmentation.** An issue reproducible only on a specific older Android OEM skin or iOS version nobody manually tested — trading apps skew toward users who care about reliability, and a crash on a less-common device is still a user who trusted the app with real funds.
- **Rounding/precision bugs that only appear at specific values** — very small trade sizes, specific decimal places, currency-conversion edges — the kind of thing a handful of manual smoke tests, run with "normal" numbers, will never surface.